

Bayescoverage APP - subnational overview

Leontine Alkema, last updated June 12, 2026

Installation info is given in the readme and shinyapp

```
library(bayescoveragedeploy)
```

Where data are stored:

```
data_folder <- "data_raw" # for survey data
routinedata_dir <- here::here("private/routinedata/")
# see note on working with routine dummy data below
```

Choose an indicator. Naming comes from survey data, options include

- c("anc1trimester", "vmsl", "ideliv", "vdpt", "anc4") , with routine as available
- c("bfexcl0_5", "ancq8", "cci", "sba"), no routine data

NOTE JUNE 12: ONLY CCI and ANC4 can be used right now

```
indicator_select <- "anc4" #"cci"
```

Read in survey data and process

```
dat0 <- haven::read_dta(here::here(data_folder, "ICEH_gregion_20260611.dta")) |>
  dplyr::rename(r = r_raw, se = se_raw, , admin1 = "level")
```

If relevant, LA added code to clean the 6/11 data set to only keep only years with same regional structure as most recent year (as some countries have different regions for different surveys).

Copying here in quarto but not used.

```

# Filter to
# For each iso-indic combination:
# 1. Find the most recent year
# 2. Get the unique regions in that year
# 3. Keep only years where regions match the most recent year
dat1 <- dat0 |>
  dplyr::group_by(iso, indic) |>
  dplyr::mutate(
    max_year = max(year, na.rm = TRUE)
  ) |>
  dplyr::group_by(iso, indic, year) |>
  dplyr::mutate(
    regions_this_year = paste(sort(unique(admin1)), collapse = "|")
  ) |>
  dplyr::group_by(iso, indic) |>
  dplyr::mutate(
    regions_max_year = regions_this_year[year == max_year][1]
  ) |>
  dplyr::filter(regions_this_year == regions_max_year) |>
  dplyr::select(-max_year, -regions_this_year, -regions_max_year) |>
  dplyr::ungroup()

```

Fitting for 1 or more regions in one country, use ISO to select

```

# countries with survey data
# unique(dat$iso)
iso_select <- "NGA"
dat_use <-
  bayescoveragemodel::process_data(
    dat = dat0 |>
    dplyr::filter(iso %in% iso_select),
    regions_dat = bayescoveragemodel::regions_all,
    indicator = indicator_select, verbose = FALSE)

```

```
[1] "Column possible_outlier not included in data, no pre-defined outliers used"
```

```

# regions
admin1s <- sort(unique(dat_use$admin1))
admin1s

```

```
[1] "Abia"           "Abuja FCT"      "Adamawa"        "Akwa ibom"
```

[5]	"Akwa Ibom"	"Akwa-Ibom"	"Anambra"	"Bauchi"
[9]	"Bayelsa"	"Benue"	"Borno"	"Cross River"
[13]	"Cross-River"	"Delta"	"Ebonyi"	"Edo"
[17]	"Ekiti"	"Enugu"	"FCT"	"FCT (Abuja)"
[21]	"FCT Abuja"	"FCT-Abuja"	"Gombe"	"Imo"
[25]	"Jigawa"	"Kaduna"	"Kano"	"Katsina"
[29]	"Kebbi"	"Kogi"	"Kwara"	"Lagos"
[33]	"Nasarawa"	"Niger"	"North central"	"North Central"
[37]	"North east"	"North East"	"North west"	"North West"
[41]	"North-Central"	"North-East"	"North-West"	"Ogun"
[45]	"Ondo"	"Osun"	"Oyo"	"Plateau"
[49]	"Rivers"	"Sokoto"	"South east"	"South East"
[53]	"South south"	"South South"	"South west"	"South West"
[57]	"South-East"	"South-South"	"South-West"	"Taraba"
[61]	"Yobe"	"Zamfara"		

Do we want to add routine data?

```
add_routine <- ifelse(
  indicator_select %in% c("anc1trimester", "vmsl", "ideliv", "vdpt", "anc4"),
  TRUE,
  FALSE)
add_routine
```

```
[1] TRUE
```

If yes, read in routine data. Make sure it has an admin1 column with admin1 names that match the survey data. DO NOT include an iso column in the routine data set.

```
if (!add_routine){
  routine_dat <- routine_dat_use <- NULL
} else {
  # use dummy data, need
  routine_dat_use <- readr::read_csv(here::here("data_raw/routine_toydata.csv")) |>
  # add admin1 column with matching names to dat_use
  dplyr::mutate(admin1 = admin1s[1],
                indicator_name = indicator_select) |>
  dplyr::select(-iso)
}
```

```

Rows: 5 Columns: 5
-- Column specification -----
Delimiter: ","
dbl (3): year, routine_value, countdownmean
lgl (2): iso, indicator_name

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

```

```
routine_dat_use
```

```

# A tibble: 5 x 5
  indicator_name year routine_value countdownmean admin1
  <chr>         <dbl>         <dbl>         <dbl> <chr>
1 anc4         2020         0.52          95 Abia
2 anc4         2021         0.53         100 Abia
3 anc4         2022         0.55         100 Abia
4 anc4         2023         0.52         100 Abia
5 anc4         2024         0.56         100 Abia

```

```

# or use a dummy set as follows:
# routine_dat_use <- data.frame(
#   iso = rep(iso_select, 3),
#   year = 2021:2023,
#   indicator_name = indicator_select,
#   routine_value = c(0.85, 0.87, 0.90),
#   countdownmean = rep(100, 3))

```

make sure to use the “subnational = TRUE” argument to fit the model. Just fitting here to the region where we have routine data

```

fit <- bayescoveragedeploy::fit_local_model(
  survey_df = dat_use |> dplyr::filter(admin1 %in% routine_dat_use$admin1),
  subnational = TRUE,
  indicator = indicator_select,
  iso_select = iso_select,
  routine_df = routine_dat_use
)

```

```

[1] "We use a global fit, and take selected settings from there."
[1] "settings for the spline_degree and num_knots taken from global run"

```

```

[1] "settings for Ptilde_low taken from global run"
[1] "Setting for tstar taken from global run"
[1] "Settings for hierarchical settings taken from global run"
[1] "For hierarchical terms, we fix things up to the 2nd-lowest level."
[1] "We fix all sigmas of hierarchical models."
[1] "We take the data model setting from global fit."
[1] "We add smoothing"
[1] "We fix smoothing and data model parameters."
[1] "We use subnational data."
[1] "We do NOT add subnational correlation."
[1] "We filter data to be inside estimation period"
[1] "We use subnational data (but you already knew that)"
[1] "We only use data for NGA"
[1] "NOT using or producing aggregates"

```

Joining with `by = join\_by(admin1)`

```

[1] "Using precompiled Stan model"

```

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 0.000266 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 2.66 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 450 [0%] (Warmup)

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 7.1e-05 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.71 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 450 [0%] (Warmup)

Chain 1: Iteration: 10 / 450 [2%] (Warmup)

Chain 2: Iteration: 10 / 450 [2%] (Warmup)

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 3).

Chain 3:

Chain 3: Gradient evaluation took 6.7e-05 seconds

Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.67 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration: 1 / 450 [0%] (Warmup)
Chain 2: Iteration: 20 / 450 [4%] (Warmup)

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 4).
Chain 1: Iteration: 20 / 450 [4%] (Warmup)
Chain 3: Iteration: 10 / 450 [2%] (Warmup)
Chain 4:
Chain 4: Gradient evaluation took 0.000176 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 1.76 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration: 1 / 450 [0%] (Warmup)
Chain 2: Iteration: 30 / 450 [6%] (Warmup)
Chain 1: Iteration: 30 / 450 [6%] (Warmup)
Chain 3: Iteration: 20 / 450 [4%] (Warmup)
Chain 2: Iteration: 40 / 450 [8%] (Warmup)
Chain 3: Iteration: 30 / 450 [6%] (Warmup)
Chain 1: Iteration: 40 / 450 [8%] (Warmup)
Chain 2: Iteration: 50 / 450 [11%] (Warmup)
Chain 1: Iteration: 50 / 450 [11%] (Warmup)
Chain 2: Iteration: 60 / 450 [13%] (Warmup)
Chain 3: Iteration: 40 / 450 [8%] (Warmup)
Chain 2: Iteration: 70 / 450 [15%] (Warmup)
Chain 1: Iteration: 60 / 450 [13%] (Warmup)
Chain 3: Iteration: 50 / 450 [11%] (Warmup)
Chain 4: Iteration: 10 / 450 [2%] (Warmup)
Chain 2: Iteration: 80 / 450 [17%] (Warmup)
Chain 1: Iteration: 70 / 450 [15%] (Warmup)
Chain 3: Iteration: 60 / 450 [13%] (Warmup)
Chain 2: Iteration: 90 / 450 [20%] (Warmup)
Chain 3: Iteration: 70 / 450 [15%] (Warmup)
Chain 1: Iteration: 80 / 450 [17%] (Warmup)
Chain 2: Iteration: 100 / 450 [22%] (Warmup)
Chain 3: Iteration: 80 / 450 [17%] (Warmup)
Chain 4: Iteration: 20 / 450 [4%] (Warmup)
Chain 1: Iteration: 90 / 450 [20%] (Warmup)
Chain 3: Iteration: 90 / 450 [20%] (Warmup)
Chain 1: Iteration: 100 / 450 [22%] (Warmup)

Chain 4: Iteration: 30 / 450 [6%] (Warmup)
 Chain 3: Iteration: 100 / 450 [22%] (Warmup)
 Chain 4: Iteration: 40 / 450 [8%] (Warmup)
 Chain 1: Iteration: 110 / 450 [24%] (Warmup)
 Chain 2: Iteration: 110 / 450 [24%] (Warmup)
 Chain 4: Iteration: 50 / 450 [11%] (Warmup)
 Chain 3: Iteration: 110 / 450 [24%] (Warmup)
 Chain 4: Iteration: 60 / 450 [13%] (Warmup)
 Chain 1: Iteration: 120 / 450 [26%] (Warmup)
 Chain 3: Iteration: 120 / 450 [26%] (Warmup)
 Chain 4: Iteration: 70 / 450 [15%] (Warmup)
 Chain 2: Iteration: 120 / 450 [26%] (Warmup)
 Chain 1: Iteration: 130 / 450 [28%] (Warmup)
 Chain 3: Iteration: 130 / 450 [28%] (Warmup)
 Chain 4: Iteration: 80 / 450 [17%] (Warmup)
 Chain 1: Iteration: 140 / 450 [31%] (Warmup)
 Chain 2: Iteration: 130 / 450 [28%] (Warmup)
 Chain 3: Iteration: 140 / 450 [31%] (Warmup)
 Chain 1: Iteration: 150 / 450 [33%] (Warmup)
 Chain 1: Iteration: 151 / 450 [33%] (Sampling)
 Chain 2: Iteration: 140 / 450 [31%] (Warmup)
 Chain 4: Iteration: 90 / 450 [20%] (Warmup)
 Chain 3: Iteration: 150 / 450 [33%] (Warmup)
 Chain 1: Iteration: 160 / 450 [35%] (Sampling)
 Chain 3: Iteration: 151 / 450 [33%] (Sampling)
 Chain 4: Iteration: 100 / 450 [22%] (Warmup)
 Chain 1: Iteration: 170 / 450 [37%] (Sampling)
 Chain 2: Iteration: 150 / 450 [33%] (Warmup)
 Chain 2: Iteration: 151 / 450 [33%] (Sampling)
 Chain 3: Iteration: 160 / 450 [35%] (Sampling)
 Chain 1: Iteration: 180 / 450 [40%] (Sampling)
 Chain 2: Iteration: 160 / 450 [35%] (Sampling)
 Chain 3: Iteration: 170 / 450 [37%] (Sampling)
 Chain 1: Iteration: 190 / 450 [42%] (Sampling)
 Chain 4: Iteration: 110 / 450 [24%] (Warmup)
 Chain 3: Iteration: 180 / 450 [40%] (Sampling)
 Chain 1: Iteration: 200 / 450 [44%] (Sampling)
 Chain 2: Iteration: 170 / 450 [37%] (Sampling)
 Chain 4: Iteration: 120 / 450 [26%] (Warmup)
 Chain 1: Iteration: 210 / 450 [46%] (Sampling)
 Chain 3: Iteration: 190 / 450 [42%] (Sampling)
 Chain Chain 2: Iteration: 180 / 450 [40%] (Sampling)
 1: Iteration: 220 / 450 [48%] (Sampling)

Chain 3: Iteration: 200 / 450 [44%] (Sampling)
Chain 1: Iteration: 230 / 450 [51%] (Sampling)
Chain 4: Iteration: 130 / 450 [28%] (Warmup)
Chain 3: Iteration: 210 / 450 [46%] (Sampling)
Chain 2: Iteration: 190 / 450 [42%] (Sampling)
Chain 1: Iteration: 240 / 450 [53%] (Sampling)
Chain 1: Iteration: 250 / 450 [55%] (Sampling)
Chain 4: Iteration: 140 / 450 [31%] (Warmup)
Chain 3: Iteration: 220 / 450 [48%] (Sampling)
Chain 2: Iteration: 200 / 450 [44%] (Sampling)
Chain 1: Iteration: 260 / 450 [57%] (Sampling)
Chain 3: Iteration: 230 / 450 [51%] (Sampling)
Chain 4: Iteration: 150 / 450 [33%] (Warmup)
Chain 4: Iteration: 151 / 450 [33%] (Sampling)
Chain 1: Iteration: 270 / 450 [60%] (Sampling)
Chain 2: Iteration: 210 / 450 [46%] (Sampling)
Chain 3: Iteration: 240 / 450 [53%] (Sampling)
Chain 1: Iteration: 280 / 450 [62%] (Sampling)
Chain 4: Iteration: 160 / 450 [35%] (Sampling)
Chain 2: Iteration: 220 / 450 [48%] (Sampling)
Chain 1: Iteration: 290 / 450 [64%] (Sampling)
Chain 3: Iteration: 250 / 450 [55%] (Sampling)
Chain 1: Iteration: 300 / 450 [66%] (Sampling)
Chain 3: Iteration: 260 / 450 [57%] (Sampling)
Chain 4: Iteration: 170 / 450 [37%] (Sampling)
Chain 2: Iteration: 230 / 450 [51%] (Sampling)
Chain 1: Iteration: 310 / 450 [68%] (Sampling)
Chain 1: Iteration: 320 / 450 [71%] (Sampling)
Chain 3: Iteration: 270 / 450 [60%] (Sampling)
Chain 4: Iteration: 180 / 450 [40%] (Sampling)
Chain 2: Iteration: 240 / 450 [53%] (Sampling)
Chain 1: Iteration: 330 / 450 [73%] (Sampling)
Chain 3: Iteration: 280 / 450 [62%] (Sampling)
Chain 1: Iteration: 340 / 450 [75%] (Sampling)
Chain 2: Iteration: 250 / 450 [55%] (Sampling)
Chain 4: Iteration: 190 / 450 [42%] (Sampling)
Chain 3: Iteration: 290 / 450 [64%] (Sampling)
Chain 1: Iteration: 350 / 450 [77%] (Sampling)
Chain 2: Iteration: 260 / 450 [57%] (Sampling)
Chain 1: Iteration: 360 / 450 [80%] (Sampling)
Chain 3: Iteration: 300 / 450 [66%] (Sampling)
Chain 4: Iteration: 200 / 450 [44%] (Sampling)
Chain 1: Iteration: 370 / 450 [82%] (Sampling)

Chain 3: Iteration: 310 / 450 [68%] (Sampling)
 Chain 2: Iteration: 270 / 450 [60%] (Sampling)
 Chain 4: Iteration: 210 / 450 [46%] (Sampling)
 Chain 1: Iteration: 380 / 450 [84%] (Sampling)
 Chain 3: Iteration: 320 / 450 [71%] (Sampling)
 Chain 1: Iteration: 390 / 450 [86%] (Sampling)
 Chain 4: Iteration: 220 / 450 [48%] (Sampling)
 Chain 2: Iteration: 280 / 450 [62%] (Sampling)
 Chain 3: Iteration: 330 / 450 [73%] (Sampling)
 Chain 1: Iteration: 400 / 450 [88%] (Sampling)
 Chain 3: Iteration: 340 / 450 [75%] (Sampling)
 Chain 4: Iteration: 230 / 450 [51%] (Sampling)
 Chain 1: Iteration: 410 / 450 [91%] (Sampling)
 Chain 2: Iteration: 290 / 450 [64%] (Sampling)
 Chain 3: Iteration: 350 / 450 [77%] (Sampling)
 Chain 1: Iteration: 420 / 450 [93%] (Sampling)
 Chain 4: Iteration: 240 / 450 [53%] (Sampling)
 Chain 1: Iteration: 430 / 450 [95%] (Sampling)
 Chain 2: Iteration: 300 / 450 [66%] (Sampling)
 Chain 3: Iteration: 360 / 450 [80%] (Sampling)
 Chain 1: Iteration: 440 / 450 [97%] (Sampling)
 Chain 3: Iteration: 370 / 450 [82%] (Sampling)
 Chain 4: Iteration: 250 / 450 [55%] (Sampling)
 Chain 2: Iteration: 310 / 450 [68%] (Sampling)
 Chain 1: Iteration: 450 / 450 [100%] (Sampling)
 Chain 1:
 Chain 1: Elapsed Time: 0.188 seconds (Warm-up)
 Chain 1: 0.203 seconds (Sampling)
 Chain 1: 0.391 seconds (Total)
 Chain 1:
 Chain 3: Iteration: 380 / 450 [84%] (Sampling)
 Chain 4: Iteration: 260 / 450 [57%] (Sampling)
 Chain 2: Iteration: 320 / 450 [71%] (Sampling)
 Chain 3: Iteration: 390 / 450 [86%] (Sampling)
 Chain 3: Iteration: 400 / 450 [88%] (Sampling)
 Chain 4: Iteration: 270 / 450 [60%] (Sampling)
 Chain 2: Iteration: 330 / 450 [73%] (Sampling)
 Chain 3: Iteration: 410 / 450 [91%] (Sampling)
 Chain 4: Iteration: 280 / 450 [62%] (Sampling)
 Chain 2: Iteration: 340 / 450 [75%] (Sampling)
 Chain 3: Iteration: 420 / 450 [93%] (Sampling)
 Chain 3: Iteration: 430 / 450 [95%] (Sampling)
 Chain 4: Iteration: 290 / 450 [64%] (Sampling)

```

Chain 2: Iteration: 350 / 450 [ 77%] (Sampling)
Chain 3: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 4: Iteration: 300 / 450 [ 66%] (Sampling)
Chain 2: Chain Iteration: 360 / 450 [ 80%] (Sampling)3: Iteration: 450 / 450 [100%] (Sampling)

Chain 3:
Chain 3: Elapsed Time: 0.163 seconds (Warm-up)
Chain 3: 0.258 seconds (Sampling)
Chain 3: 0.421 seconds (Total)
Chain 3:
Chain 4: Iteration: 310 / 450 [ 68%] (Sampling)
Chain 2: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 4: Iteration: 320 / 450 [ 71%] (Sampling)
Chain 2: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 4: Iteration: 330 / 450 [ 73%] (Sampling)
Chain 2: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 4: Chain Iteration: 340 / 450 [ 75%] (Sampling)
2: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 4: Iteration: 350 / 450 [ 77%] (Sampling)
Chain 2: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 4: Iteration: 360 / 450 [ 80%] (Sampling)
Chain 2: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 2: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 4: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 2: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 4: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 2: Iteration: 450 / 450 [100%] (Sampling)
Chain 4: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 0.188 seconds (Warm-up)
Chain 2: 0.367 seconds (Sampling)
Chain 2: 0.555 seconds (Total)
Chain 2:
Chain 4: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 4: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 4: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 4: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 4: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 4: Iteration: 450 / 450 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 0.217 seconds (Warm-up)
Chain 4: 0.379 seconds (Sampling)
Chain 4: 0.596 seconds (Total)

```

Chain 4:

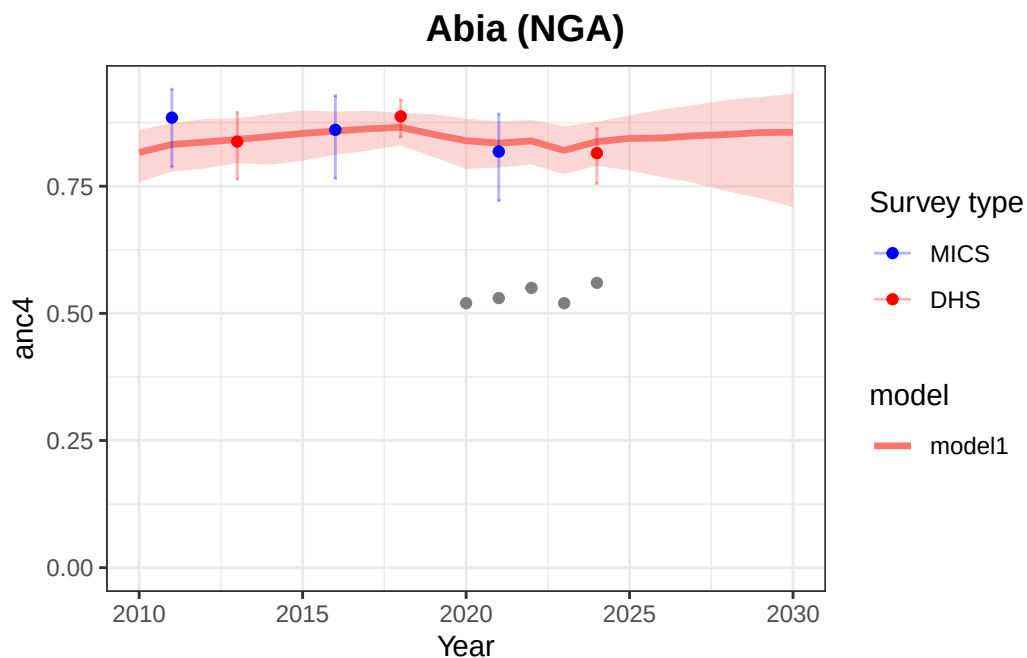
Warning: There were 3 divergent transitions after warmup. See <https://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup> to find out why this is a problem and how to eliminate them.

Warning: Examine the pairs() plot to diagnose sampling problems

Extracting posteriors...

Overview plot

```
countryplot <- bayescoveragemodel::plot_estimates_local_all(  
  results = fit,  
  save_plots = FALSE)  
countryplot[[1]]
```



Estimates

```
fit$posteriors$temporal |>  
  dplyr::select(iso, admin1, year, `50%`, `2.5%`, `97.5%`)
```

```
# A tibble: 21 x 6
  iso   admin1  year `50%` `2.5%` `97.5%`
  <chr> <chr>   <int> <dbl> <dbl> <dbl>
1 NGA   Abia     2010 0.816 0.758 0.861
2 NGA   Abia     2011 0.832 0.779 0.874
3 NGA   Abia     2012 0.837 0.785 0.882
4 NGA   Abia     2013 0.842 0.795 0.883
5 NGA   Abia     2014 0.848 0.792 0.892
6 NGA   Abia     2015 0.854 0.800 0.899
7 NGA   Abia     2016 0.858 0.812 0.897
8 NGA   Abia     2017 0.863 0.820 0.898
9 NGA   Abia     2018 0.866 0.830 0.894
10 NGA   Abia     2019 0.852 0.807 0.891
# i 11 more rows
```

Compare to a fit w/o routine data, all regions here

```
fit_noroutine<- bayescoveragedeploy::fit_local_model(
  # fit just for the first region
  survey_df = dat_use,
  subnational = TRUE,
  indicator = indicator_select,
  iso_select = iso_select,
  routine_df = NULL)
```

```
[1] "We use a global fit, and take selected settings from there."
[1] "settings for the spline_degree and num_knots taken from global run"
[1] "settings for Ptilde_low taken from global run"
[1] "Setting for tstar taken from global run"
[1] "Settings for hierarchical settings taken from global run"
[1] "For hierarchical terms, we fix things up to the 2nd-lowest level."
[1] "We fix all sigmas of hierarchical models."
[1] "We take the data model setting from global fit."
[1] "We add smoothing"
[1] "We fix smoothing and data model parameters."
[1] "We use subnational data."
[1] "We do NOT add subnational correlation."
[1] "We filter data to be inside estimation period"
[1] "We use subnational data (but you already knew that)"
[1] "We only use data for NGA"
[1] "NOT using or producing aggregates"
[1] "Using precompiled Stan model"
```

SAMPLING FOR MODEL 'fpem' NOW (CHAIN 1).

SAMPLING FOR MODEL 'fpem' NOW (CHAIN 2).

Chain 1:

Chain 1: Gradient evaluation took 0.001274 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 12.74 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

SAMPLING FOR MODEL 'fpem' NOW (CHAIN 3).

Chain 1: Iteration: 1 / 450 [0%] (Warmup)

Chain 2:

Chain 2: Gradient evaluation took 0.016752 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 167.52 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 450 [0%] (Warmup)

SAMPLING FOR MODEL 'fpem' NOW (CHAIN 4).

Chain 3:

Chain 3: Gradient evaluation took 0.002281 seconds

Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 22.81 seconds.

Chain 3: Adjust your expectations accordingly!

Chain 3:

Chain 3:

Chain 3: Iteration: 1 / 450 [0%] (Warmup)

Chain 4:

Chain 4: Gradient evaluation took 0.001003 seconds

Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 10.03 seconds.

Chain 4: Adjust your expectations accordingly!

Chain 4:

Chain 4:

Chain 4: Iteration: 1 / 450 [0%] (Warmup)

Chain 1: Iteration: 10 / 450 [2%] (Warmup)

Chain 4: Iteration: 10 / 450 [2%] (Warmup)

Chain 1: Iteration: 20 / 450 [4%] (Warmup)

Chain 4: Iteration: 20 / 450 [4%] (Warmup)

Chain 3: Iteration: 10 / 450 [2%] (Warmup)

Chain 4: Iteration: 30 / 450 [6%] (Warmup)

Chain 1: Iteration: 30 / 450 [6%] (Warmup)

Chain 4: Iteration: 40 / 450 [8%] (Warmup)
 Chain 1: Iteration: 40 / 450 [8%] (Warmup)
 Chain 3: Iteration: 20 / 450 [4%] (Warmup)
 Chain 1: Iteration: 50 / 450 [11%] (Warmup)
 Chain 3: Iteration: 30 / 450 [6%] (Warmup)
 Chain 1: Iteration: 60 / 450 [13%] (Warmup)
 Chain 4: Iteration: 50 / 450 [11%] (Warmup)
 Chain 3: Iteration: 40 / 450 [8%] (Warmup)
 Chain 1: Iteration: 70 / 450 [15%] (Warmup)
 Chain 3: Iteration: 50 / 450 [11%] (Warmup)
 Chain 4: Iteration: 60 / 450 [13%] (Warmup)
 Chain 3: Iteration: 60 / 450 [13%] (Warmup)
 Chain 1: Iteration: 80 / 450 [17%] (Warmup)
 Chain 3: Iteration: 70 / 450 [15%] (Warmup)
 Chain 4: Iteration: 70 / 450 [15%] (Warmup)
 Chain 1: Iteration: 90 / 450 [20%] (Warmup)
 Chain 1: Iteration: 100 / 450 [22%] (Warmup)
 Chain 4: Iteration: 80 / 450 [17%] (Warmup)
 Chain 3: Iteration: 80 / 450 [17%] (Warmup)
 Chain 2: Iteration: 10 / 450 [2%] (Warmup)
 Chain 3: Iteration: 90 / 450 [20%] (Warmup)
 Chain 4: Iteration: 90 / 450 [20%] (Warmup)
 Chain 1: Iteration: 110 / 450 [24%] (Warmup)
 Chain 4: Iteration: 100 / 450 [22%] (Warmup)
 Chain 1: Iteration: 120 / 450 [26%] (Warmup)
 Chain 3: Iteration: 100 / 450 [22%] (Warmup)
 Chain 1: Iteration: 130 / 450 [28%] (Warmup)
 Chain 4: Iteration: 110 / 450 [24%] (Warmup)
 Chain 3: Iteration: 110 / 450 [24%] (Warmup)
 Chain 3: Iteration: 120 / 450 [26%] (Warmup)
 Chain 1: Iteration: 140 / 450 [31%] (Warmup)
 Chain 3: Iteration: 130 / 450 [28%] (Warmup)
 Chain 1: Iteration: 150 / 450 [33%] (Warmup)
 Chain 1: Iteration: 151 / 450 [33%] (Sampling)
 Chain 1: Iteration: 160 / 450 [35%] (Sampling)
 Chain 3: Iteration: 140 / 450 [31%] (Warmup)
 Chain 1: Iteration: 170 / 450 [37%] (Sampling)
 Chain 3: Iteration: 150 / 450 [33%] (Warmup)
 Chain 3: Iteration: 151 / 450 [33%] (Sampling)
 Chain 1: Iteration: 180 / 450 [40%] (Sampling)
 Chain 3: Iteration: 160 / 450 [35%] (Sampling)
 Chain 1: Iteration: 190 / 450 [42%] (Sampling)
 Chain 4: Iteration: 120 / 450 [26%] (Warmup)

Chain 3: Iteration: 170 / 450 [37%] (Sampling)
Chain 1: Iteration: 200 / 450 [44%] (Sampling)
Chain 3: Iteration: 180 / 450 [40%] (Sampling)
Chain 1: Iteration: 210 / 450 [46%] (Sampling)
Chain 3: Iteration: 190 / 450 [42%] (Sampling)
Chain 1: Iteration: 220 / 450 [48%] (Sampling)
Chain 3: Iteration: 200 / 450 [44%] (Sampling)
Chain 1: Iteration: 230 / 450 [51%] (Sampling)
Chain 4: Iteration: 130 / 450 [28%] (Warmup)
Chain 3: Iteration: 210 / 450 [46%] (Sampling)
Chain 1: Iteration: 240 / 450 [53%] (Sampling)
Chain 3: Iteration: 220 / 450 [48%] (Sampling)
Chain 1: Iteration: 250 / 450 [55%] (Sampling)
Chain 3: Iteration: 230 / 450 [51%] (Sampling)
Chain 1: Iteration: 260 / 450 [57%] (Sampling)
Chain 3: Iteration: 240 / 450 [53%] (Sampling)
Chain 1: Iteration: 270 / 450 [60%] (Sampling)
Chain 3: Iteration: 250 / 450 [55%] (Sampling)
Chain 4: Iteration: 140 / 450 [31%] (Warmup)
Chain 1: Iteration: 280 / 450 [62%] (Sampling)
Chain 3: Iteration: 260 / 450 [57%] (Sampling)
Chain 1: Iteration: 290 / 450 [64%] (Sampling)
Chain 3: Iteration: 270 / 450 [60%] (Sampling)
Chain 1: Iteration: 300 / 450 [66%] (Sampling)
Chain 3: Iteration: 280 / 450 [62%] (Sampling)
Chain 1: Iteration: 310 / 450 [68%] (Sampling)
Chain 3: Iteration: 290 / 450 [64%] (Sampling)
Chain 1: Iteration: 320 / 450 [71%] (Sampling)
Chain 4: Iteration: 150 / 450 [33%] (Warmup)
Chain 3: Iteration: 300 / 450 [66%] (Sampling)
Chain 4: Iteration: 151 / 450 [33%] (Sampling)
Chain 1: Iteration: 330 / 450 [73%] (Sampling)
Chain 3: Iteration: 310 / 450 [68%] (Sampling)
Chain 2: Iteration: 20 / 450 [4%] (Warmup)
Chain 1: Iteration: 340 / 450 [75%] (Sampling)
Chain 3: Iteration: 320 / 450 [71%] (Sampling)
Chain 1: Iteration: 350 / 450 [77%] (Sampling)
Chain 3: Iteration: 330 / 450 [73%] (Sampling)
Chain 1: Iteration: 360 / 450 [80%] (Sampling)
Chain 4: Iteration: 160 / 450 [35%] (Sampling)
Chain 3: Iteration: 340 / 450 [75%] (Sampling)
Chain 1: Iteration: 370 / 450 [82%] (Sampling)
Chain 2: Iteration: 30 / 450 [6%] (Warmup)

```

Chain 3: Iteration: 350 / 450 [ 77%] (Sampling)
Chain 1: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 2: Iteration:  40 / 450 [  8%] (Warmup)
Chain 3: Iteration: 360 / 450 [ 80%] (Sampling)
Chain 1: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 2: Iteration:  50 / 450 [ 11%] (Warmup)
Chain 2: Iteration:  60 / 450 [ 13%] (Warmup)
Chain 3: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 1: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 4: Iteration: 170 / 450 [ 37%] (Sampling)
Chain 3: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 1: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 3: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 1: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 3: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 1: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 2: Iteration:  70 / 450 [ 15%] (Warmup)
Chain 3: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 1: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 4: Iteration: 180 / 450 [ 40%] (Sampling)
Chain 3: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 1: Iteration: 450 / 450 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 45.269 seconds (Warm-up)
Chain 1:                73.817 seconds (Sampling)
Chain 1:                119.086 seconds (Total)
Chain 1:
Chain 2: Iteration:  80 / 450 [ 17%] (Warmup)
Chain 3: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 2: Iteration:  90 / 450 [ 20%] (Warmup)
Chain 3: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 2: Iteration: 100 / 450 [ 22%] (Warmup)
Chain 3: Iteration: 450 / 450 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 51.827 seconds (Warm-up)
Chain 3:                73.924 seconds (Sampling)
Chain 3:                125.751 seconds (Total)
Chain 3:
Chain 2: Iteration: 110 / 450 [ 24%] (Warmup)
Chain 4: Iteration: 190 / 450 [ 42%] (Sampling)
Chain 2: Iteration: 120 / 450 [ 26%] (Warmup)
Chain 2: Iteration: 130 / 450 [ 28%] (Warmup)
Chain 2: Iteration: 140 / 450 [ 31%] (Warmup)

```



```

Chain 2: Iteration: 150 / 450 [ 33%] (Warmup)
Chain 2: Iteration: 151 / 450 [ 33%] (Sampling)
Chain 4: Iteration: 200 / 450 [ 44%] (Sampling)
Chain 2: Iteration: 160 / 450 [ 35%] (Sampling)
Chain 2: Iteration: 170 / 450 [ 37%] (Sampling)
Chain 2: Iteration: 180 / 450 [ 40%] (Sampling)
Chain 2: Iteration: 190 / 450 [ 42%] (Sampling)
Chain 4: Iteration: 210 / 450 [ 46%] (Sampling)
Chain 2: Iteration: 200 / 450 [ 44%] (Sampling)
Chain 2: Iteration: 210 / 450 [ 46%] (Sampling)
Chain 2: Iteration: 220 / 450 [ 48%] (Sampling)
Chain 2: Iteration: 230 / 450 [ 51%] (Sampling)
Chain 4: Iteration: 220 / 450 [ 48%] (Sampling)
Chain 2: Iteration: 240 / 450 [ 53%] (Sampling)
Chain 2: Iteration: 250 / 450 [ 55%] (Sampling)
Chain 2: Iteration: 260 / 450 [ 57%] (Sampling)
Chain 2: Iteration: 270 / 450 [ 60%] (Sampling)
Chain 4: Iteration: 230 / 450 [ 51%] (Sampling)
Chain 2: Iteration: 280 / 450 [ 62%] (Sampling)
Chain 2: Iteration: 290 / 450 [ 64%] (Sampling)
Chain 2: Iteration: 300 / 450 [ 66%] (Sampling)
Chain 2: Iteration: 310 / 450 [ 68%] (Sampling)
Chain 4: Iteration: 240 / 450 [ 53%] (Sampling)
Chain 2: Iteration: 320 / 450 [ 71%] (Sampling)
Chain 2: Iteration: 330 / 450 [ 73%] (Sampling)
Chain 2: Iteration: 340 / 450 [ 75%] (Sampling)
Chain 2: Iteration: 350 / 450 [ 77%] (Sampling)
Chain 4: Iteration: 250 / 450 [ 55%] (Sampling)
Chain 2: Iteration: 360 / 450 [ 80%] (Sampling)
Chain 2: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 2: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 2: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 4: Iteration: 260 / 450 [ 57%] (Sampling)
Chain 2: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 2: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 2: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 2: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 4: Iteration: 270 / 450 [ 60%] (Sampling)
Chain 2: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 2: Iteration: 450 / 450 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 136.152 seconds (Warm-up)
Chain 2: 72.36 seconds (Sampling)

```

```

Chain 2:                208.512 seconds (Total)
Chain 2:
Chain 4: Iteration: 280 / 450 [ 62%] (Sampling)
Chain 4: Iteration: 290 / 450 [ 64%] (Sampling)
Chain 4: Iteration: 300 / 450 [ 66%] (Sampling)
Chain 4: Iteration: 310 / 450 [ 68%] (Sampling)
Chain 4: Iteration: 320 / 450 [ 71%] (Sampling)
Chain 4: Iteration: 330 / 450 [ 73%] (Sampling)
Chain 4: Iteration: 340 / 450 [ 75%] (Sampling)
Chain 4: Iteration: 350 / 450 [ 77%] (Sampling)
Chain 4: Iteration: 360 / 450 [ 80%] (Sampling)
Chain 4: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 4: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 4: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 4: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 4: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 4: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 4: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 4: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 4: Iteration: 450 / 450 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 89.273 seconds (Warm-up)
Chain 4:                286.231 seconds (Sampling)
Chain 4:                375.504 seconds (Total)
Chain 4:

```

Warning: There were 5 divergent transitions after warmup. See <https://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup> to find out why this is a problem and how to eliminate them.

Warning: Examine the pairs() plot to diagnose sampling problems

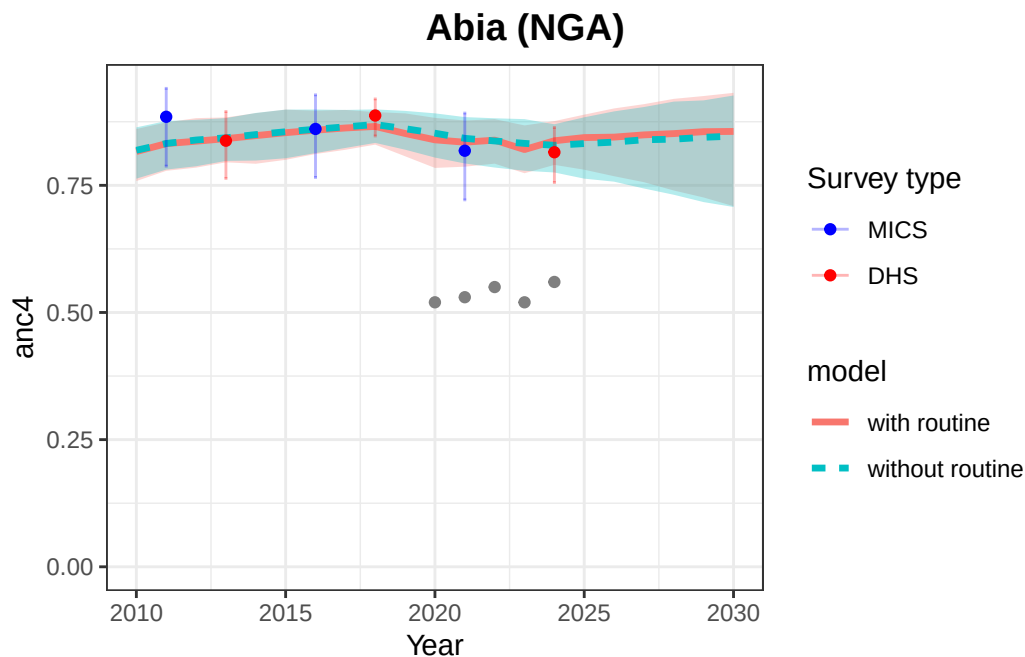
Warning: The largest R-hat is NA, indicating chains have not mixed. Running the chains for more iterations may help. See <https://mc-stan.org/misc/warnings.html#r-hat>

Warning: Bulk Effective Samples Size (ESS) is too low, indicating posterior means and medians may not be accurate. Running the chains for more iterations may help. See <https://mc-stan.org/misc/warnings.html#bulk-ess>

Warning: Tail Effective Samples Size (ESS) is too low, indicating posterior variances and ta
Running the chains for more iterations may help. See
<https://mc-stan.org/misc/warnings.html#tail-ess>

Extracting posteriors...

```
countryplot <- bayescoveragemodel::plot_estimates_local_all(
  results = fit,
  results2 = fit_noroutine,
  region_codes = routine_dat_use$admin1[1],
  modelnames = c("with routine", "without routine"),
  save_plots = FALSE)
countryplot[[1]]
```



Add comparison plot

```
p <- bayescoveragemodel::plot_subnational_comparison(results = fit,
  results2 = fit_noroutine,
  model_names = c("Model w routine data",
    "Model w/o routine data"),
  year_select = 2025)
p
```

